

LAPORAN TEKNIS

CDCC358D6Y0569 - Mohammad Wildan Abdurrahman
CDCC358D6Y2763 - Ahmad Fajar Permadi

BAB I

PROBLEM DISCOVERY

1.1 Latar Belakang dan Urgensi Bisnis

Pasar tenaga kerja saat ini bergerak dengan sangat dinamis dan menghasilkan volume data lowongan yang luar biasa besar di berbagai platform digital, salah satunya LinkedIn. Sayangnya, limpahan data ini sering kali hadir dalam bentuk yang berantakan dan tidak terstruktur. Di sisi lain, para pencari kerja kerap kali kebingungan dan mengalami blind spot (titik buta) mengenai kualifikasi mereka sendiri. Mereka sulit mengukur secara pasti seberapa besar kesenjangan keahlian (skill-gap) yang mereka miliki dibandingkan dengan kebutuhan riil di dunia industri.

Berangkat dari masalah tersebut, proyek CareerPath AI dikembangkan sebagai solusi cerdas berbasis data. Proyek ini dirancang untuk menarik data dari pasar kerja, membersihkannya, melakukan eksplorasi mendalam, hingga akhirnya mampu memetakan kebutuhan kompetensi industri secara presisi. Namun, agar sistem dashboard dan model kecerdasan buatan (AI) di dalam platform ini dapat memberikan rekomendasi yang akurat, kebersihan data mentah dan ketajaman analisis di tahap awal menjadi kunci yang paling menentukan.

1.2 Rumusan Masalah Bisnis

1. Pekerjaan apa saja yang paling sering muncul berdasarkan data pada dataset?
2. Kota atau wilayah mana saja yang memiliki peluang kerja paling banyak?
3. Bagaimana distribusi tipe pekerjaan (seperti Full-time, Contract, atau Internship) beserta jumlah lowongannya ?
4. Bagaimana distribusi tingkat pengalaman yang dicari oleh perusahaan beserta jumlah lowongannya?
5. Pada periode apa proses rekrutmen lebih aktif dari biasanya?
6. Kompetensi di bidang apa yang paling sering muncul berdasarkan data pada dataset?
7. Jenis keahlian spesifik apa yang wajib dikuasai jika merujuk pada 5 besar lowongan kerja terpopuler?
8. Bagaimana perbandingan antara syarat pengalaman kerja dengan jumlah lowongan yang tersedia nyata di dataset?
9. Apakah lokasi geografis tempat pekerjaan berada memberikan pengaruh yang signifikan terhadap keahlian yang mendominasi?

BAB II

ASSESSING DATA

2.1 Temuan masalah kualitas data dan evidence

1. Syntactic HTML Noise : Fitur deskripsi lowongan (description) terkontaminasi oleh tag enkapsulasi antarmuka web LinkedIn.
 - Evidence: Teks korpus pada indeks awal menunjukkan tumpukan tag struktural: "...Are you a smart, authentic...
ROLE AND RESPONSIBILITIESWin, retain, and develop business accounts..."

```
[2] Cek Anomali Tag HTML (Noise)
import re

def has_html(text):
    pattern = re.compile('<.*>') # Mencari pola <...tag...>
    return bool(pattern.search(str(text)))

print(f"--- [2] Anomali Noise (HTML) ---")
anomali_html = df[df['description'].apply(has_html)]

print(f"Data dengan sisa tag HTML: {len(anomali_html)} data")
if not anomali_html.empty:
    display(anomali_html[['title', 'description']].head(5))
```

```
[16]
--- [2] Anomali Noise (HTML) ---
Data dengan sisa tag HTML: 2 data
```

	title	description
2556	Staff cost control	Hallo Everyone, Sushi Stop adalah restaurant J...
3623	Business Executive Allianz	🔵 We're Hiring – Business Executive 🔵 Tired of ...

2. Ketidakstandaran Judul: Fitur nama jabatan (title) tidak murni karena membawa teks dekoratif, rate upah, dan simbol pipa penanda lokasi.
 - Evidence: Hasil pengecekan data menunjukkan teks tidak baku seperti "Software Engineer | Remote @ \$75/hr".

	title	location
490	Integration Software Engineer \$80 Remote	APJ
521	Fire Service Supervisor \$75/hr Remote	APJ
673	Senior AI Product Platform Engineer @ High Gro...	Denpasar, Bali, Indonesia
745	LibreSprite Pixel Artist – Game Art & Animatio...	APJ
748	Pixel Artist -LibreSprite (Animation & Sprites...	APJ
790	Pixel Art Animator \$65/hr Remote	APJ
1206	HR Policy & Compliance Specialist \$36/hr Remote	APJ
1259	Audio Engineer (Audio QA & Restoration) \$55/...	APJ
1361	HR Policy & Compliance Specialist \$36/hr Remote	Indonesia
1485	Motion Data Collection Specialist- Video & Sen...	APJ
1493	Motion Data Collection Specialist- Video & Sen...	Indonesia
1538	Madurese Language Specialist \$95 Remote	Indonesia
1550	Madurese Translator \$95/hr Remote	Indonesia
1558	Document Specialist - Word and PDF \$65/hr Re...	APJ
1567	Document Specialist - Word and PDF \$65/hr Re...	Indonesia
2707	Digital Illustrator (Krita) \$65/hr Remote	APJ
2718	Digital Illustrator (Krita) \$65/hr Remote	Indonesia
2742	STEM Expert - Fully Remote Upto \$16/hr	Jakarta, Jakarta, Indonesia
3427	Junior Software Engineer (TypeScript) \$42/hr...	APJ
3428	Junior Software Engineer (TypeScript) \$42/hr...	Indonesia

3. Ambiguitas Lokasi Geografis : Fitur lokasi (location) mencatat cakupan wilayah makro-regional yang terlalu luas dan tidak spesifik bagi pencari kerja lokal.
- Evidence: Banyak data berlabel cakupan benua seperti 'APAC', 'APJ', 'Southeast Asia', atau sekadar 'Indonesia'.

```
print(f"Total Lokasi Unik: {df['location'].nunique()}")
# Menampilkan 30 lokasi terpopuler
print(df['location'].value_counts().head(30))
```

Python

Total Lokasi Unik: 308

location	
Jakarta, Indonesia	867
Jakarta Metropolitan Area	458
Jakarta, Jakarta, Indonesia	355
Indonesia	234
APAC	207
Tangerang, Banten, Indonesia	136
South Jakarta, Jakarta, Indonesia	81
Surabaya, East Java, Indonesia	79
Bandung, West Java, Indonesia	77
Batam, Riau Islands, Indonesia	76
Denpasar, Bali, Indonesia	72
Bali, Indonesia	63
APJ	61
Gambir, Jakarta, Indonesia	56
Semarang, Central Java, Indonesia	43
Bogor, West Java, Indonesia	40
Lembang, West Java, Indonesia	40
Yogyakarta, Yogyakarta, Indonesia	30
Southeast Asia	25
West Jakarta, Jakarta, Indonesia	24
Palembang, South Sumatra, Indonesia	23
Greater Semarang	23
West Karawang, West Java, Indonesia	23
...	
Yaur, Papua, Indonesia	17
Surakarta, Central Java, Indonesia	16
North Jakarta, Jakarta, Indonesia	16

Name: count, dtype: int64

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

keterangan Indonesia sebaiknya dihapus, karena hampir 90% data berada di Indonesia untuk wilayah (South, North, dsb) diubah ke Bahasa Indonesia (Selatan, Utara, dsb, dan Jakarta Metropolitan Area diubah menjadi istilah yang lebih familiar, yaitu Jabodetabek.

4. Missing Values Ekstrem: Fitur tingkat pengalaman (formatted_experience_level) mengalami kekosongan data yang sangat masif akibat tidak diisinya kualifikasi tersebut oleh pihak rekruter.
- Evidence: Hasil kalkulasi fungsi statistik deskriptif membuktikan tingkat kekosongan data mencapai 72%.

```
# Menampilkan kategori level pengalaman
print(df['formatted_experience_level'].unique())
print("\nJumlah per kategori (termasuk yang kosong):")
print(df['formatted_experience_level'].value_counts(dropna=False))
```

Python

[nan 'Mid-Senior level' 'Entry level' 'Associate' 'Internship' 'Director' 'Executive']

Jumlah per kategori (termasuk yang kosong):

formatted_experience_level	
NaN	2947
Mid-Senior level	539
Associate	315
Entry level	178
Director	51
Internship	33
Executive	22

Name: count, dtype: int64

Terdapat 2.947 data kosong (NaN). Ini artinya sekitar 70% lowongan tidak mencantumkan tingkat pengalaman secara formal. data kosong nantinya diisi dengan tidak terdefinisi

5. Data Administratif Kosong : Kolom penunjang operasional aplikasi seperti company_id dan application_url terdeteksi memiliki nilai null/missing.

Evidence: Terdapat baris data kosong (NaN) pada kolom ID korporasi dan URL ke lowongan.

```
# Mengecek total missing values dan persentasenya
missing_values = df.isnull().sum()
missing_percentage = (missing_values / len(df)) * 100

# Menggabungkan hasil dalam satu DataFrame
missing_df = pd.DataFrame({
    'Total Missing': missing_values,
    'Percentage Missing (%)': missing_percentage
})

# Menampilkan hanya kolom yang memiliki missing values, diurutkan dari yang terbanyak
missing_df = missing_df[missing_df['Total Missing'] > 0].sort_values(by='Total Missing', ascending=False)

# Menampilkan hasil
missing_df
```

	Total Missing	Percentage Missing (%)
skills_desc	4011	98.188494
formatted_experience_level	2947	72.141983
application_url	2252	55.128519
company_id	57	1.395349
description	10	0.244798

6. Anomali Bahasa Multinasional : banyak ditemukan penggunaan bahasa global diluar Bahasa Indonesia dan Bahasa Inggris di dataset.
 - Evidence: ditemukan 62 data lowongan yang menggunakan bahasa asing (seperti Mandarin, Belanda, dan Portugis).
 - o Kode program :

```
!pip install langdetect
from langdetect import detect
import time

def check_lang_combined(row):
    cols_to_check = [
        'title', 'description', 'skills_desc', 'location',
        'formatted_work_type', 'formatted_experience_level'
    ]

    # Gabungkan semua isi kolom tersebut menjadi satu string panjang
    combined_text = ""
    for col in cols_to_check:
        if col in row and not pd.isna(row[col]):
            combined_text += str(row[col]) + " "

    try:
        # Jika string kosong setelah digabung, return unknown
        if not combined_text.strip():
            return "unknown"
        # Deteksi bahasa dari gabungan teks tersebut
        return detect(combined_text)
    except:
        return "unknown"

print("--- [3] Memulai Deteksi Bahasa Global (Seluruh Teks & Semua Kolom) ---")
print("Proses ini akan mengecek Judul, Lokasi, Deskripsi, hingga Level secara bersamaan...")

start_time = time.time()

# Menjalankan deteksi pada gabungan kolom per baris (axis=1)
df['detected_lang_global'] = df.apply(check_lang_combined, axis=1)

end_time = time.time()
durasi = end_time - start_time

print(f"Selesai dalam {durasi:.2f} detik.")

# 1. Tampilkan Statistik Bahasa
print("\nStatistik Bahasa yang Terdeteksi secara Global:")
print(df['detected_lang_global'].value_counts())

# 2. Filter Anomali (Selain Indonesia 'id' dan Inggris 'en')
anomali_global = df[df['detected_lang_global'].isin(['id', 'en'])]

print("\n--- HASIL ANOMALI BAHASA GLOBAL ---")
print(f"Jumlah data anomali ditemukan: {len(anomali_global)}")

if not anomali_global.empty:
    # Tampilkan kolom-kolom utama untuk pengecekan
    display(anomali_global[['title', 'location', 'detected_lang_global', 'description']])
else:
    print("Sempurna! Seluruh kolom di semua baris hanya berisi Bahasa Indonesia atau Inggris.")
```

- o Hasil :

```

Installing setuptools 1.8.0 for py (380 KB)
...
Building wheel for setuptools (setuptools)
...
Successfully installed setuptools
...
Installing collected packages: setuptools
Successfully installed setuptools-1.8.0
...
[1] Memulai Deteksi Bahasa Kiblat (Seluruh Teks & Semua Kiblat)
Proses ini akan menghapus Teks, Lokasi, dan Waktu, hingga level secara bersamaan...

Selanjut dalam 30-35 detik.

Statistik Bahasa yang Terdeteksi secara Global:
detected_lang_global
en      1000
id       700
en-es    50
en       10
...
Name: count, dtype: int64

...
[2] Memulai Deteksi Kiblat.
Terdapat data anomali ditemukan: 62
Output is truncated. View it as a jupyterlab or open it as a text editor. Adjust cell output settings...

```

BAB III DATA CLEANING

3.1 Membersihkan sampah sintaksis web/HTML

- Langkah Pembersihan: Untuk membersihkan sisa-sisa tag enkapsulasi antarmuka web LinkedIn yang mengotori deskripsi kerja (seperti potongan tag `</strong>`), saya merancang fungsi pembersihan `clean_html(text)` berbasis ekspresi reguler `re.compile('<.*?>')`. Fungsi ini diterapkan ke seluruh baris kolom deskripsi menggunakan metode `.apply()`.
- Evidence : pengecekan ulang setelah pembersihan menggunakan fungsi deteksi `has_html` membuktikan bahwa data yang memiliki sisa tag HTML sudah tidak ada 0 data.

```

import re

def clean_html(text):
    clean = re.compile('<.*?>')
    return re.sub(clean, '', str(text))

df.loc[:, 'description'] = df['description'].apply(clean_html)

print("Tag HTML berhasil dihapus.")

-- Tag HTML berhasil dihapus secara permanen.

cek apakah data dengan tag html sudah hilang

import re
def has_html(text):
    pattern = re.compile('<.*?>') # Mencari pola <...tag...>
    return bool(pattern.search(str(text)))

anomali_html = df[df['description'].apply(has_html)]
print("Data dengan sisa tag HTML: {len(anomali_html)} data")
if not anomali_html.empty:
    display(anomali_html[['title', 'description']].head(5))

-- Data dengan sisa tag HTML: 0 data

```

3.2 Membersihkan judul pekerjaan dari Noise finansial

- Langkah Pembersihan: Untuk membersihkan 20 judul pekerjaan yang terkontaminasi oleh nominal gaji, simbol mata uang, dan rate upah per jam, saya menerapkan metode pembersihan berbasis ekspresi reguler (Regular Expression). Menggunakan pola `salary_pattern = r'(\$|Rp|s?d+|IDR?s?d+|)'`, sistem menghapus string-string transaksional tersebut. Setelah itu, sisa simbol pipa (`|`) atau garis miring (`/`) di ujung string dirapikan menggunakan `.str.strip()` agar menyisakan nama jabatan murni.

- ```
1. Menangani Inkonsistensi Data

salary_pattern = r'^(?![0-9])\s*(?!\s?(($[RpIDR](USD)?[A-Z_]{1,4})?[\s?]{1,4})$'
df['title'] = df['title'].str.replace(salary_pattern, '', regex=True).str.strip()
df['title'] = df['title'].str.replace(r'[//]$', '', regex=True).str.strip()
cek apa masih ada judul yang memiliki karakter @, $, Rp, IDR
currency_pattern = r'^(($\s\d+|Rp\s\d+|IDR\s\d+)$'
title_with_salary = df[df['title'].str.contains(currency_pattern, regex=True, na=False)]
print(f"Total ditemukan: {len(title_with_salary)} data{n}")

if not title_with_salary.empty:
 # Menampilkan judul agar kita bisa lihat polanya
 display(title_with_salary[['title', 'location']])
else:
 print("Data Bersih")

Total ditemukan: 0 data

Data Bersih
/tmp/ipykernel_11510/1621548691.py:6: UserWarning: This pattern is interpreted as a regular expression, and has match groups. To actually get the groups, use str.extract.
title_with_salary = df[df['title'].str.contains(currency_pattern, regex=True, na=False)]
```

```
Mengubah Lokasi Indonesia, APAC, dan APJ menjadi remote. Karena, lokasi kurang spesifik/jelas.
remote_keywords = ['Indonesia', 'APAC', 'APJ', 'Southeast Asia']
df['location'] = df['location'].apply(lambda x: 'Remote/Nasional' if x in remote_keywords else x)
```

```
... Total Missing Percentage Missing (%)
skills_desc 4011 98.188494
formatted_experience_level 2947 72.141983
application_url 2252 55.128519
company_id 57 1.395349
description 10 0.244798
```

Missing Values Sangat Tinggi (>50%) -> Hapus

Jika suatu kolom memiliki lebih dari 50% data yang hilang, maka kemungkinan besar informasi di dalamnya tidak cukup berguna untuk analisis.

Syarat:

Kolom memiliki missing values lebih dari 50% dari total data. Kolom tidak bisa diimputasi dengan cara yang masuk akal. Kolom tidak memiliki dampak signifikan terhadap analisis, karena data skill sudah tersedia di kolom description

Tindakan:

Hapus kolom tersebut dari dataset. Kolom yang Dihapus:

skill\_desc (98.18% missing)

```
[20]: df.drop(columns=['skills_desc'], inplace=True, errors='ignore')
 print(df.columns.tolist())

... ['job_id', 'company_id', 'title', 'description', 'formatted_work_type', 'location', 'job_posting_url', 'application_url', 'formatted_experience_level', 'work_type', 'detected_lang_global']
```

### 3.5 Melakukan imputasi kolom URL dan identitas Perusahaan

- Langkah Pembersihan: Untuk mengamankan kolom penunjang operasional aplikasi dari nilai kosong (NaN), dilakukan tindakan imputasi taktis:
  1. Mengisi data kosong sebanyak 55,12% pada kolom tautan eksternal lamaran (application\_url) dengan label teks konstan 'No URL Provided' menggunakan fungsi .fillna().

|                            | Total Missing | Percentage Missing (%) |
|----------------------------|---------------|------------------------|
| skills_desc                | 4011          | 98.188494              |
| formatted_experience_level | 2947          | 72.141983              |
| application_url            | 2252          | 55.128519              |

Melakukan imputasi pada kolom application\_url "No URL Provided"

```
df['application_url'] = df['application_url'].fillna('No URL Provided')
```

2. Mengisi data kosong sebanyak 1,39% pada kolom identitas korporasi (company\_id) dengan nilai default numerik 0.

```
Mengisi company_id yang kosong dengan 0
df['company_id'] = df['company_id'].fillna(0)
```

- Langkah Pembersihan: Kolom description hanya memiliki tingkat kekosongan sebesar 0,24% (10 data). Namun, karena ini merupakan pilar data paling vital bagi proyek CareerPath AI, baris yang kosong ini langsung dibuang secara permanen dari dataset menggunakan perintah df.dropna(subset=['description'], inplace=True).

|             |    |          |
|-------------|----|----------|
| description | 10 | 0.244798 |
|-------------|----|----------|

### 3.6 Melakukan penyaringan pada outlier Bahasa global

- Langkah Pembersihan: Untuk membuang 62 data lowongan yang membawa dokumen lintas bahasa asing (seperti Mandarin, Belanda, dan Portugis) di luar fokus platform, saya melakukan pemfilteran ketat menggunakan library langdetect. Dataset disaring hanya untuk mempertahankan baris yang mendeteksi Bahasa Indonesia (id) dan Bahasa Inggris (en) lewat perintah: df = df[df['detected\_lang\_global'].isin(['id', 'en'])]. terdapat 1 data yang terdeteksi unknown, namun jika dicek manual, itu adalah gabungan bahasa Indonesia dan Inggris didalam satu baris
- Evidence :



```
1. Menghapus data yang tidak menggunakan Bahasa Indonesia atau Inggris

df = df[df['detected_lang_global'].isin(['id', 'en'])]

from langdetect import detect
import time

def check_lang_combined(row):
 col_to_check = ['title', 'description', 'skills_desc', 'location',
 'forwarded_job_type', 'forwarded_experience_level']
 # Gabungkan semua isi kolom tersebut menjadi satu string panjang
 combined_text = ''
 for col in col_to_check:
 if col in row and not pd.isna(row[col]):
 combined_text += str(row[col]) + " "
 try:
 # Jika string memang sudah digabung, return unknown
 if not combined_text.strip():
 return "unknown"
 # Deteksi bahasa dari gabungan teks tersebut
 return detect(combined_text)
 except:
 return "unknown"

print("--- [1] Memulai Deteksi Bahasa Global (seluruh Teks & Semua Kolom) ---")
print("Proses ini akan mengambil Judul, Lokasi, Deskripsi, hingga level secara bersamaan...")
start_time = time.time()

Menjalankan deteksi pada gabungan kolom per baris (axis=1)
df['detected_lang_global'] = df.apply(check_lang_combined, axis=1)

end_time = time.time()
durasi = end_time - start_time

print(f"Selesai dalam (durasi: {df.durasi}")

1. Tampilkan Statistik Bahasa
print("Statistik Bahasa yang Terdeteksi secara Global:")
print(df['detected_lang_global'].value_counts())

2. Filter Anomali (selain Indonesia 'id' dan Inggris 'en')
anomaly_global = df[df['detected_lang_global'].isin(['id', 'en'])]

print(f"----- HASIL ANOMALI BAHASA GLOBAL -----")
print("Terdapat data anomali ditemukan: {len(anomaly_global)}")

if not anomaly_global.empty:
 # Tampilkan kolom-kolom utama untuk pengamatan
 display(anomaly_global[['title', 'location', 'detected_lang_global', 'description']])
else:
 print("Semua data seluruh baris di semua baris hanya berisi Bahasa Indonesia atau Inggris.")

Requirement already satisfied: langdetect in /usr/local/lib/python3.11/dist-packages (1.0.9)
Requirement already satisfied: six in /usr/local/lib/python3.11/dist-packages (from langdetect) (1.17.0)
--- [1] Memulai Deteksi Bahasa Global (seluruh Teks & Semua Kolom) ---
Proses ini akan mengambil Judul, Lokasi, Deskripsi, hingga level secara bersamaan...

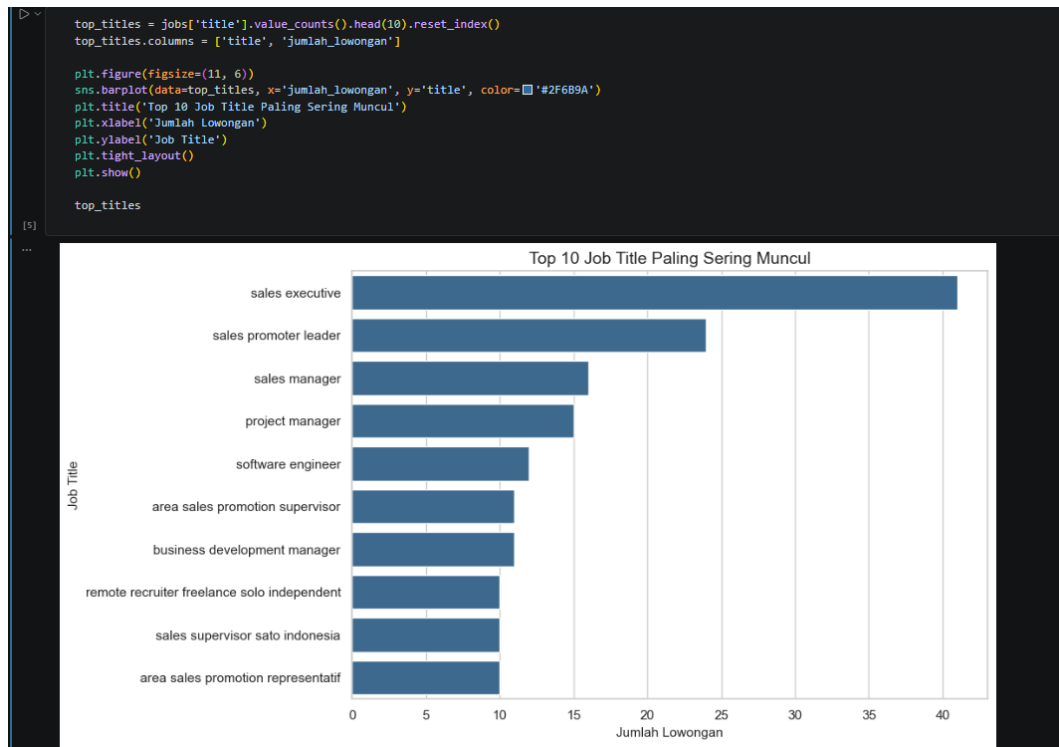
Selesai dalam 36.81 detik.

Statistik Bahasa yang Terdeteksi secara Global:
detected_lang_global
en 2279
id 744
unknown 1
Name: count, dtype: int64
```

## BAB IV EXPLORATORY DATA ANALYSIS

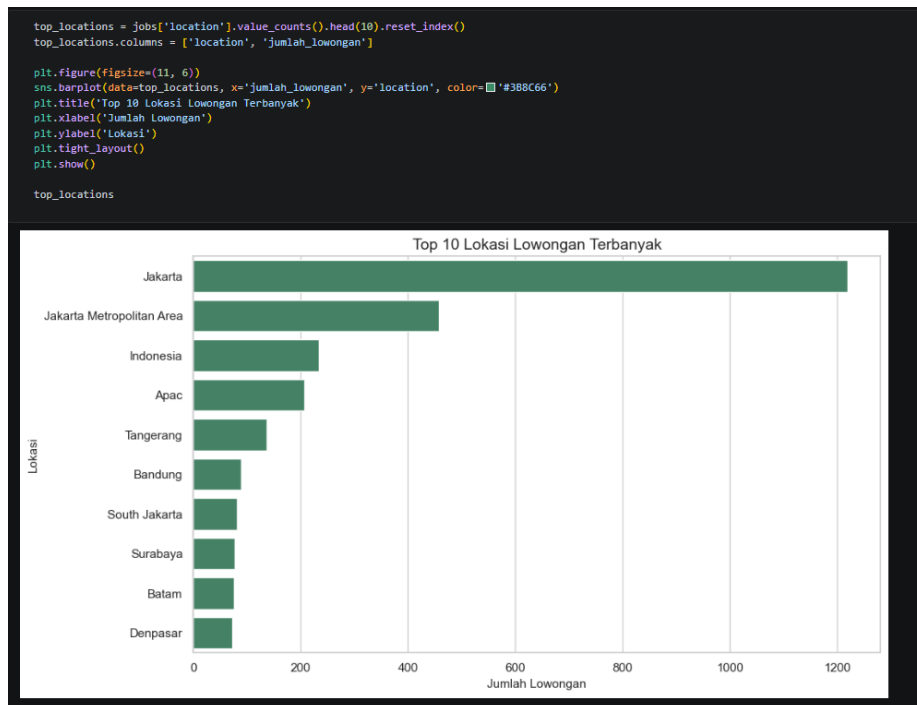
### 4.1 Analisis dominasi pekerjaan

- Langkah Analisis: kami menyisir data nama jabatan pada kolom title menggunakan fungsi `.value_counts().head(10)` dan memvisualisasikannya ke dalam bentuk grafik batang horizontal (horizontal bar chart) agar mudah dibaca.
- Evidence : Hasilnya cukup menarik, posisi Sales Executive memimpin pasar kerja dengan angka mencolok, yaitu 41 lowongan. Di belakangnya menyusul Sales Promoter Leader (24 lowongan), Sales Manager (16 lowongan), Project Manager (15 lowongan), dan posisi teknis seperti Software Engineer (12 lowongan).



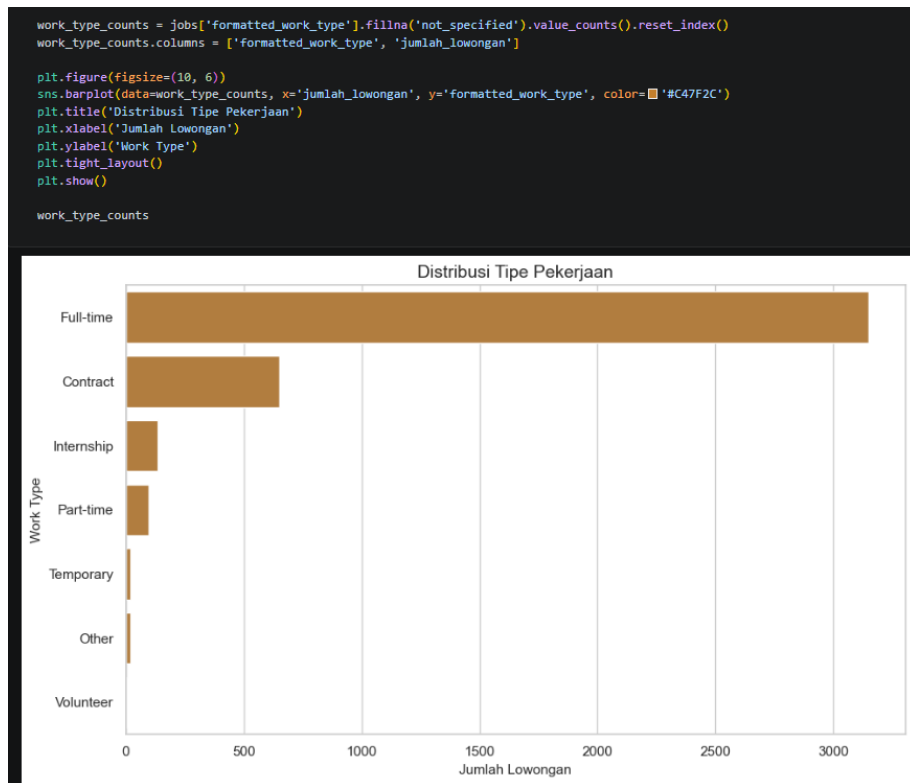
#### 4.2 Analisis sebaran geografis peluang kerja

- Langkah Analisis: Saya memetakan konsentrasi lowongan kerja berdasarkan variabel location untuk melihat kota mana yang paling aktif membuka lowongan.
- Evidence (Bukti Data): Data memperlihatkan keterpusatan wilayah yang sangat timpang. Wilayah Jakarta menjadi magnet utama dengan 1.219 lowongan. Angka ini diperkuat oleh wilayah penunjang seperti Jakarta Metropolitan Area (457 lowongan), serta label wilayah makro seperti Indonesia (234 lowongan) dan APAC (207 lowongan).



### 4.3 Analisis karakteristik tipe kontrak kerja

- Langkah Analisis: Saya membedah kolom formatted\_work\_type untuk memahami preferensi sistem kontrak kerja yang paling sering ditawarkan oleh perusahaan.
- Evidence (Bukti Data): Industri saat ini masih didominasi secara masif oleh status pekerjaan Full-time (Penuh Waktu) dengan total 3.149 lowongan. Opsi kontrak lain berada jauh di bawahnya, seperti Contract (650 lowongan), Internship (134 lowongan), dan Part-time (96 lowongan).



#### 4.4 Analisis pola syarat pengalaman industri

- Langkah Analisis: Saya mengaudit kolom `formatted_experience_level` untuk melihat seberapa detail perusahaan mencantumkan syarat level pengalaman kerja untuk menjawab pertanyaan bisnis nomor 4.
- Evidence : Di sini ditemukan sebuah anomali. Mayoritas perusahaan ternyata tidak mengisi level pengalaman kerja secara formal pada kolom bawaan LinkedIn, di mana kategori `not_specified` sebanyak 2.937 lowongan atau sekitar 72%. Bagi lowongan yang diisi secara jelas, posisi Mid-Senior level memimpin dengan 539 lowongan, diikuti Associate (315 lowongan), dan Entry level (178 lowongan).

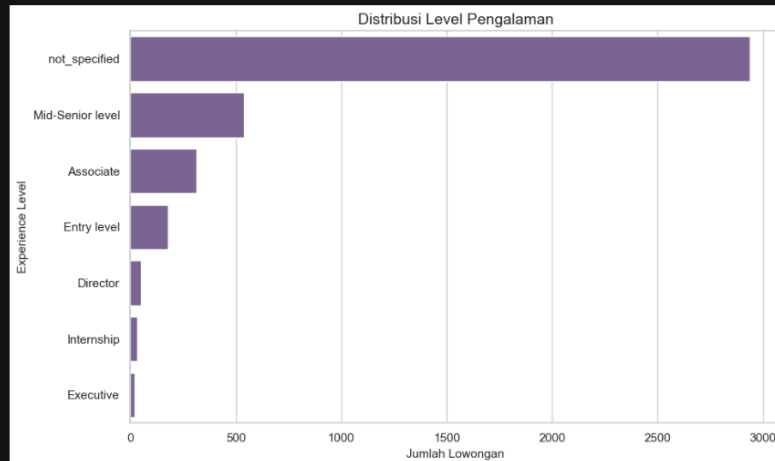
#### D. Experience Level

Analisis level pengalaman membantu memahami apakah pasar lebih banyak membutuhkan fresh graduate, junior, mid-level, senior, atau kandidat dengan pengalaman tidak dispesifikasikan.

```
experience_counts = jobs['formatted_experience_level'].fillna('not_specified').value_counts().reset_index()
experience_counts.columns = ['formatted_experience_level', 'jumlah_lowongan']

plt.figure(figsize=(10, 6))
sns.barplot(data=experience_counts, x='jumlah_lowongan', y='formatted_experience_level', color='#7A5C99')
plt.title('Distribusi Level Pengalaman')
plt.xlabel('Jumlah Lowongan')
plt.ylabel('Experience Level')
plt.tight_layout()
plt.show()

experience_counts
```



#### 4.5 Analisis tren waktu aktivitas rekrutmen

- Langkah Analisis: Saya mengubah data string tanggal menjadi tipe datetime standar (`pd.to_datetime()`), lalu mengelompokkannya berdasarkan tren bulanan (`.dt.to_period('M')`) untuk melihat grafik garis rekrutmen dari waktu ke waktu.
- Evidence (Bukti Data): Lonjakan aktivitas rekrutmen terpantau sangat agresif dan menumpuk pada periode Bulan April 2026, yang mencatatkan total 4.075 lowongan aktif diposting dalam waktu yang bersamaan.

### E. Tren Waktu Posting

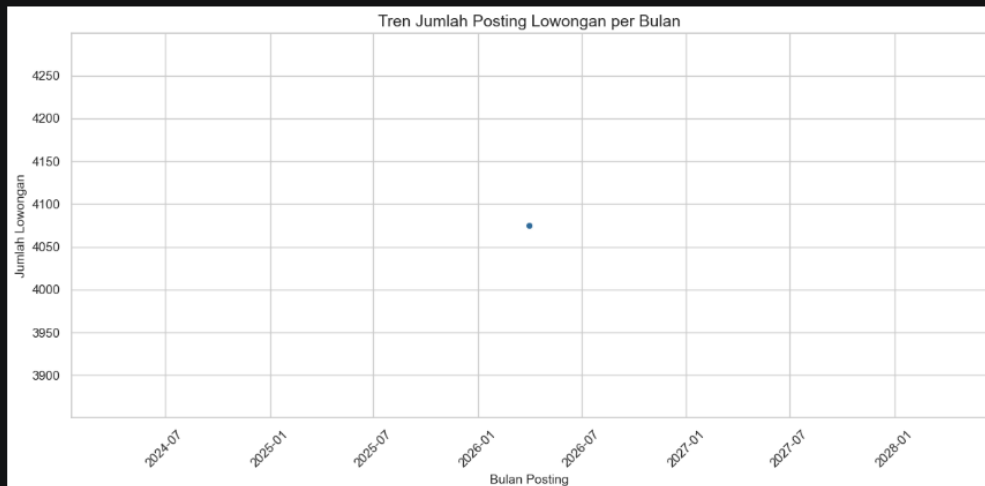
Tren posting dianalisis berdasarkan `listed_time`. Jika kolom ini belum bertipe `datetime`, kolom akan dikonversi terlebih dahulu.

```
jobs['listed_time'] = pd.to_datetime(jobs['listed_time'], errors='coerce')
jobs['original_listed_time'] = pd.to_datetime(jobs['original_listed_time'], errors='coerce')

jobs['posting_month'] = jobs['listed_time'].dt.to_period('M').dt.to_timestamp()
monthly_postings = jobs.dropna(subset=['posting_month']).groupby('posting_month').size().reset_index(name='jumlah_lowongan')

plt.figure(figsize=(12, 6))
sns.lineplot(data=monthly_postings, x='posting_month', y='jumlah_lowongan', marker='o', color='#2F689A')
plt.title('Tren Jumlah Posting Lowongan per Bulan')
plt.xlabel('Bulan Posting')
plt.ylabel('Jumlah Lowongan')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

monthly_postings
```



### Integrasi komparatif data keahlian

- Langkah Analisis: Untuk mendapatkan analisis keahlian (skills) yang tajam, saya melakukan penggabungan data. Pertama, tabel relasi `job_skills` diperkaya dengan master data nama skill dari tabel `skills`. Setelah itu, hasilnya digabungkan kembali secara inner join dengan dataset lowongan bersih (`jobs`) berbasis variabel `job_id`. Proses ini menghasilkan sebuah dataset baru sebanyak 6.844 baris data.

#### ~ A. Merge Data Skill

Proses merge dilakukan dalam dua tahap:

1. `job_skills` digabung dengan `skills` berdasarkan `skill_abr` untuk mendapatkan `skill_name`.
2. Hasil tersebut digabung dengan `job_clean_final` berdasarkan `job_id` agar setiap skill memiliki konteks pekerjaan, lokasi, dan experience level.

```
skill_mapping = job_skills.merge(skills, on='skill_abr', how='left')

skill_analysis = skill_mapping.merge(
 jobs[['job_id', 'title', 'location', 'formatted_experience_level', 'formatted_work_type']],
 on='job_id',
 how='inner'
)

skill_analysis = skill_analysis[[
 'job_id',
 'title',
 'skill_abr',
 'skill_name',
 'location',
 'formatted_experience_level',
 'formatted_work_type'
]].copy()

print(f'Shape hasil merge skill: {skill_analysis.shape[0]:,} baris x {skill_analysis.shape[1]:,} kolom')
print(f'Jumlah skill tanpa nama setelah merge: {skill_analysis["skill_name"].isna().sum():,}')
skill_analysis.head()
```

```
[18]
... Shape hasil merge skill: 6,844 baris x 7 kolom
Jumlah skill tanpa nama setelah merge: 0
```

#### 4.6 Melakukan analisis kompetensi bidang yang paling dicari

- Langkah Analisis: Saya menghitung total frekuensi kemunculan kata kunci keahlian pada kolom hasil gabungan data (`skill_name`).
- Evidence : Kompetensi di bidang Sales (Penjualan) menjadi keahlian paling populer di industri saat ini dengan total kemunculan 1.061 kali. Posisi kedua ditempati oleh bidang Information Technology / IT (841 kali), disusul oleh Business Development (654 kali), Engineering (485 kali), dan Marketing (399 kali).

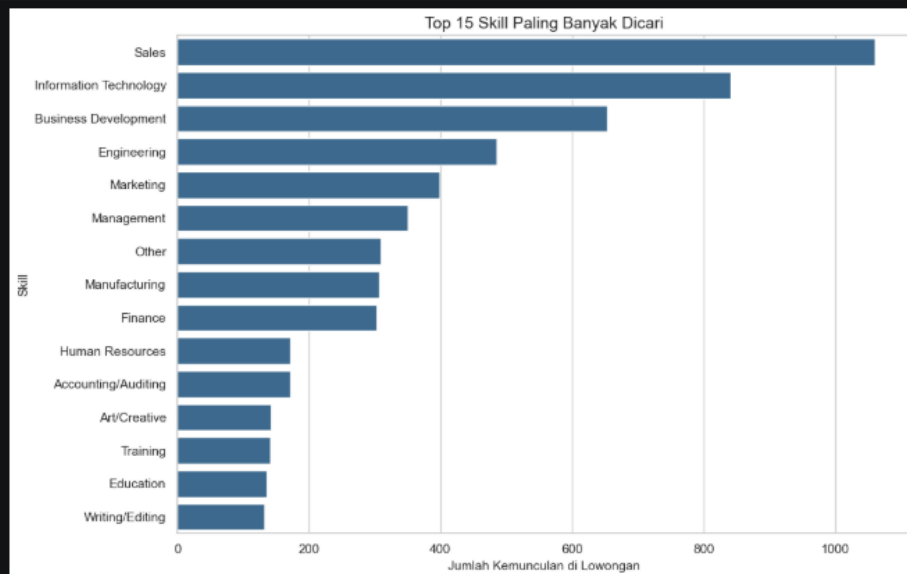
## B. Top 15 Skill Paling Dicari

Frekuensi skill menunjukkan kompetensi yang paling banyak muncul di lowongan kerja.

```
top_skills = skill_analysis['skill_name'].value_counts().head(15).reset_index()
top_skills.columns = ['skill_name', 'jumlah_lowongan']

plt.figure(figsize=(11, 7))
sns.barplot(data=top_skills, x='jumlah_lowongan', y='skill_name', color='#2F6B9A')
plt.title("Top 15 Skill Paling Banyak Dicari")
plt.xlabel('Jumlah Kemunculan di Lowongan')
plt.ylabel('Skill')
plt.tight_layout()
plt.show()

top_skills
```



|    | skill_name             | jumlah_lowongan |
|----|------------------------|-----------------|
| 0  | Sales                  | 1061            |
| 1  | Information Technology | 841             |
| 2  | Business Development   | 654             |
| 3  | Engineering            | 485             |
| 4  | Marketing              | 399             |
| 5  | Management             | 351             |
| 6  | Other                  | 309             |
| 7  | Manufacturing          | 307             |
| 8  | Finance                | 303             |
| 9  | Human Resources        | 172             |
| 10 | Accounting/Auditing    | 172             |
| 11 | Art/Creative           | 143             |
| 12 | Training               | 141             |
| 13 | Education              | 136             |

### 4.7 Melakukan Analisa spesifik keahlian pada 5 lowongan teratas

- Langkah Analisis: Saya mengisolasi data khusus untuk 5 besar nama jabatan tertinggi di dataset, kemudian mengelompokkannya menggunakan fungsi `.groupby(['title', 'skill_name'])` untuk melihat jenis keahlian inti yang wajib dikuasai pada masing-masing posisi.
- Evidence : Analisis ini membuktikan bahwa syarat keahlian industri bersifat sangat spesifik dan melekat erat pada jenis pekerjaannya.
  - o Program



```

top_5_titles = jobs['title'].value_counts().head(5).index.tolist()

skills_by_title = (
 skill_analysis[skill_analysis['title'].isin(top_5_titles)]
 .groupby(['title', 'skill_name'])
 .size()
 .reset_index(name='jumlah_lowongan')
 .sort_values(['title', 'jumlah_lowongan'], ascending=[True, False])
)

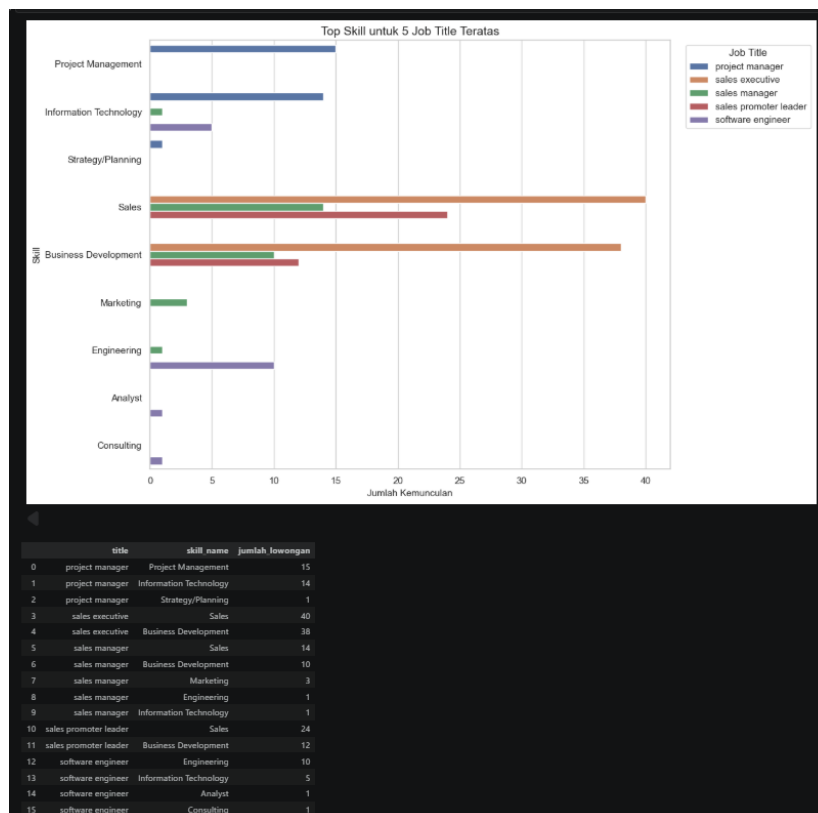
top_skills_by_title = skills_by_title.groupby('title').head(5).reset_index(drop=True)

plt.figure(figsize=(13, 8))
sns.barplot(data=top_skills_by_title, x='jumlah_lowongan', y='skill_name', hue='title')
plt.title('Top Skill untuk 5 Job Title Teratas')
plt.xlabel('Jumlah Kemunculan')
plt.ylabel('Skill')
plt.legend(title='Job Title', bbox_to_anchor=(1.02, 1), loc='upper left')
plt.tight_layout()
plt.show()

top_skills_by_title

```

○ Hasil



#### 4.8 Melakukan Analisa pengaruh Lokasi terhadap dominasi keahlian

- Langkah Analisis: Saya menyaring data berdasarkan 3 besar lokasi penempatan kerja tertinggi (Jakarta, Jakarta Metropolitan Area, Indonesia) untuk melihat apakah faktor letak geografis memengaruhi jenis keahlian yang dicari.
- Evidence (Bukti Data): Ditemukan pola dominasi geografis yang sangat kontras. Di wilayah megapolitan seperti Jakarta dan sekitarnya, kebutuhan kompetensi bergeser dari yang tadinya bersifat teknis murni menjadi keahlian strategis-komersial, seperti Sales, Information Technology, dan Business Development. Hal ini terjadi karena wilayah tersebut menjadi pusat operasional kantor pusat korporasi.

○ Program :

```
def categorize_experience_level(level):
 level = str(level).lower()
 if 'intern' in level:
 return 'intern'
 if 'entry' in level or 'associate' in level:
 return 'junior'
 if 'senior' in level or 'director' in level or 'executive' in level or 'mid' in level:
 return 'senior'
 return 'other'

skill_analysis['experience_group'] = skill_analysis['formatted_experience_level'].apply(categorize_experience_level)

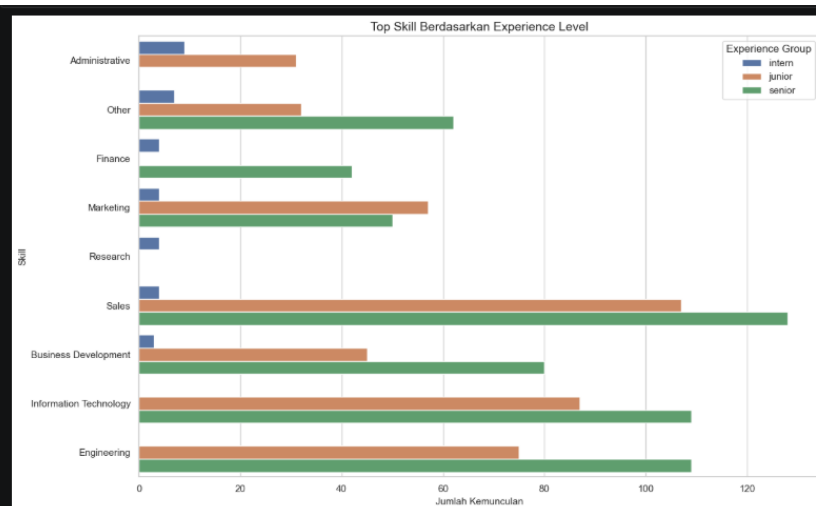
skill_by_experience = (
 skill_analysis[skill_analysis['experience_group'].isin(['junior', 'senior', 'intern'])]
 .groupby(['experience_group', 'skill_name'])
 .size()
 .reset_index(name='jumlah_lowongan')
 .sort_values(['experience_group', 'jumlah_lowongan'], ascending=[True, False])
)

top_skill_by_experience = skill_by_experience.groupby('experience_group').head(7).reset_index(drop=True)

plt.figure(figsize=(13, 8))
sns.barplot(data=top_skill_by_experience, x='jumlah_lowongan', y='skill_name', hue='experience_group')
plt.title('Top Skill Berdasarkan Experience Level')
plt.xlabel('Jumlah Kemunculan')
plt.ylabel('Skill')
plt.legend(title='Experience Group')
plt.tight_layout()
plt.show()

top_skill_by_experience
```

○ Hasil :



| experience_group | skill_name             | jumlah_lowongan |
|------------------|------------------------|-----------------|
| intern           | Administrative         | 9               |
| intern           | Other                  | 7               |
| intern           | Finance                | 4               |
| intern           | Marketing              | 4               |
| intern           | Research               | 4               |
| intern           | Sales                  | 4               |
| intern           | Business Development   | 3               |
| junior           | Sales                  | 107             |
| junior           | Information Technology | 87              |
| junior           | Engineering            | 75              |
| junior           | Marketing              | 57              |
| junior           | Business Development   | 45              |
| junior           | Other                  | 32              |
| junior           | Administrative         | 31              |
| senior           | Sales                  | 128             |
| senior           | Engineering            | 109             |
| senior           | Information Technology | 109             |
| senior           | Business Development   | 80              |
| senior           | Other                  | 62              |
| senior           | Marketing              | 50              |
| senior           | Finance                | 42              |

Insight: Perbedaan skill antar level pengalaman dapat membantu CareerPath AI memberikan rekomendasi yang lebih personal. Untuk fresh graduate atau junior, sistem dapat menekankan skill dasar yang sering muncul, sedangkan untuk senior sistem dapat menambahkan skill manajerial, teknis lanjut, atau domain-specific.

## BAB V

### FEATURE ENGINEERING

#### 5.1 Validasi awal

- Langkah dan Alasan: Proses diawali dengan memuat dataset hasil pembersihan (`job_clean_final.csv`) dan data agregat kompetensi murni (`skill_analysis_ready.csv`) menggunakan library Pandas dan objek Path dari `pathlib`. Langkah audit awal dilakukan secara otomatis menggunakan fungsi kustom `audit_data()` untuk memvalidasi ketersediaan kolom kunci seperti `job_id`, `title`, `description`, `location`, dan `listed_time`.
- Evidence Data: Program mengevaluasi nilai duplikasi menggunakan perintah `jobs.duplicated().sum()` dan menghasilkan 0 duplikat baris serta memastikan kolom data esensial industri siap digunakan.

Validasi awal dilakukan untuk memahami missing values, duplikasi, dan tipe data sebelum feature engineering. Dataset siap untuk feature engineering jika kolom utama seperti `job_id`, `title`, `description`, `location`, dan `listed_time` tersedia.

```
def audit_data(data, dataset_name):
 return pd.DataFrame({
 'dataset': dataset_name,
 'kolom': data.columns,
 'tipe_data': data.dtypes.astype(str).values,
 'missing_count': data.isna().sum().values,
 'missing_percent': (data.isna().mean() * 100).round(2).values,
 'unique_values': data.nunique(dropna=False).values
 })

audit_jobs = audit_data(jobs, 'job_clean_final')
display(audit_jobs.sort_values('missing_percent', ascending=False))

print(f'Duplicate rows job_clean_final: {jobs.duplicated().sum():,}')
if 'job_id' in jobs.columns:
 print(f'Duplicate job_id job_clean_final: {jobs.duplicated(subset=["job_id"]).sum():,}')

if skill_ready_available:
 audit_skill = audit_data(skill_ready, 'skill_analysis_ready')
 display(audit_skill.sort_values('missing_percent', ascending=False))
 print(f'Duplicate rows skill_analysis_ready: {skill_ready.duplicated().sum():,}')
```

#### 5.2 Feature Engineering Dasar

##### a. Title Clean

- Langkah dan Alasan: Judul pekerjaan (`title`) mentah dirapikan menjadi fitur baru bernama `title_clean` melalui fungsi transformasi `clean_simple_text()`. Langkah ini mengubah string ke dalam huruf kecil (`lowercase`), menggunakan Regular Expression `['^a-z0-9\s\+\#\.\.']` untuk menyeleksi karakter teks dan angka murni dengan tetap mempertahankan karakter pemrograman sensitif seperti tanda `+` dan `#` (untuk skill seperti `C++` atau `C#`), serta mereduksi spasi ganda (`whitespace normalization`)
- Evidence

### A. Title Clean

Kolom `title_clean` dibuat dari `title` dengan lowercase, penghapusan simbol, dan whitespace normalization.

```
def clean_simple_text(text):
 """Membersihkan teks dasar untuk fitur NLP dan matching."""
 if pd.isna(text):
 return ''
 text = str(text).lower()
 text = re.sub(r'^a-z0-9\s\+\#\.\|', ' ', text)
 text = re.sub(r'\s+', ' ', text).strip()
 return text

jobs['title_clean'] = jobs['title'].apply(clean_simple_text)
jobs[['title', 'title_clean']].head()
```

Python

### b. Description Clean

- Langkah dan Alasan: Dengan menggunakan fungsi logika yang sama, kolom teks deskripsi kerja (`description`) ditransformasikan menjadi variabel baru bernama `description_clean`. Evidence Data: Seluruh teks kualifikasi industri diubah menjadi teks bersih yang homogen, siap diumpungkan sebagai asupan data numerik kontekstual untuk model arsitektur NLP Deep Learning.
- Evidence :

### B. Description Clean

Kolom `description_clean` dibuat dengan merapikan teks dan menghapus spasi ganda. Fitur ini menjadi input penting untuk NLP dan semantic matching.

```
jobs['description_clean'] = jobs['description'].apply(clean_simple_text)
jobs[['description', 'description_clean']].head()
```

Python

### c. Description Length

- Langkah dan Alasan: Saya merancang ekspresi fungsi lambda berupa `len(x.split()) if x else 0` untuk menghitung total kata bersih dari kolom `description_clean` dan menyimpannya ke dalam variabel numerik baru bernama `desc_length`.
- Evidence Data: Fitur ini bertindak sebagai indikator densitas dan kompleksitas syarat kerja. Deskripsi yang panjang memberikan kuantitas informasi tugas dan kualifikasi keahlian yang jauh lebih kaya bagi pemodelan teks semantic

`desc_length` berisi jumlah kata pada `description_clean`. Secara teknis, deskripsi yang lebih panjang sering menunjukkan informasi pekerjaan yang lebih detail dan kompleks.

```
jobs['desc_length'] = jobs['description_clean'].apply(lambda x: len(x.split()) if x else 0)
jobs[['title_clean', 'desc_length']].head()
```

Python

**Insight teknis:** `desc_length` dapat digunakan sebagai fitur kualitas informasi lowongan. Lowongan dengan deskripsi sangat pendek mungkin kurang informatif untuk model NLP, sedangkan deskripsi yang panjang biasanya menyediakan sinyal tugas, skill, dan kualifikasi yang lebih kaya.

### d. Is Remote

- Langkah Analisis: Untuk menyaring fleksibilitas tempat kerja, kolom `remote_allowed` diubah menjadi tipe data numerik yang aman menggunakan fungsi `pd.to_numeric(errors='coerce').fillna(0).astype(int)`. Dari hasil tersebut,

dirancang kondisi boolean menggunakan fungsi NumPy np.where() untuk melahirkan fitur biner mutlak: is\_remote.

- Evidence Data: Nilai 1 menunjukkan lowongan mengizinkan sistem kerja jarak jauh (remote allowed), sedangkan nilai 0 menandakan status non-remote atau tidak terdefinisi

#### D. Is Remote

is\_remote dibuat dari remote\_allowed. Nilai 1 berarti remote diperbolehkan, sedangkan 0 berarti non-remote atau tidak diketahui.

```
jobs['is_remote'] = pd.to_numeric(jobs.get('remote_allowed', 0), errors='coerce').fillna(0).astype(int)
jobs['is_remote'] = np.where(jobs['is_remote'] == 1, 1, 0)

jobs[['remote_allowed', 'is_remote']].head() if 'remote_allowed' in jobs.columns else jobs[['is_remote']].head()
```

Python

#### e. Experience Level Clean

- Langkah Analisis: Untuk menangani masalah 72% data kualifikasi tingkat pengalaman bawaan LinkedIn yang kosong (not\_specified), saya menyusun fungsi penalaran cerdas bernama normalize\_experience\_level(). Fungsi ini menyisir teks gabungan antara formatted\_experience\_level dengan title\_clean untuk mendeteksi kata kunci tersirat seperti 'intern', 'junior', 'associate', 'senior', 'manager', atau 'lead'. Jika tidak ditemukan kata kunci, sistem secara otomatis memberikan nilai tengah pasar berupa label 'mid'
- Evidence :

#### E. Experience Level Clean

experience\_level\_clean distandarisasi dari formatted\_experience\_level. Jika kosong atau not\_specified, notebook mengekstrak sinyal dari title\_clean, seperti junior, senior, intern, associate, manager, dan lead. Jika tidak ada sinyal, nilai default adalah mid.

```
def normalize_experience_level(level, title):
 """Menstandarisasi experience level dari kolom level dan title lowongan."""
 level_text = '' if pd.isna(level) else str(level).lower().strip()
 title_text = '' if pd.isna(title) else str(title).lower().strip()
 combined_text = level_text + ' ' + title_text

 if 'intern' in combined_text or 'internship' in combined_text:
 return 'intern'
 if 'junior' in combined_text or 'entry' in combined_text:
 return 'junior'
 if 'associate' in combined_text:
 return 'associate'
 if 'manager' in combined_text or 'mr.' in combined_text:
 return 'manager'
 if 'senior' in combined_text:
 return 'senior'
 if 'lead' in combined_text or 'head' in combined_text or 'director' in combined_text or 'executive' in combined_text:
 return 'lead'
 if level_text and level_text not in ['not_specified', 'nan', 'none', '']:
 if 'mid' in level_text:
 return 'mid'
 return 'mid'

experience_source = jobs['formatted_experience_level'] if 'formatted_experience_level' in jobs.columns else pd.Series("", index=jobs.index)
jobs['experience_level_clean'] = [
 normalize_experience_level(level, title)
 for level, title in zip(experience_source, jobs['title_clean'])
]

jobs[['formatted_experience_level', 'title_clean', 'experience_level_clean']].head() if 'formatted_experience_level' in jobs.columns else jobs[['title_clean', 'experience_level_clean']].head()
```

### 5.3 Feature Engineering Skill

#### a. Total Job per Skill

total\_skills menghitung jumlah skill unik untuk setiap lowongan.

- Menggunakan basis data dari tabel skill\_analysis\_ready.csv, saya mengelompokkan keahlian industri per pekerjaan (groupby('job\_id')) dan menerapkan fungsi agregasi .agg() menggunakan metode penghitungan unik ('skill\_name\_clean', 'nunique'). Langkah ini menghasilkan variabel numerik baru bernama total\_skills. Fitur ini berfungsi memberikan informasi kuantitatif mengenai jumlah total syarat keahlian dari setiap lowongan kerja.

#### b. Dominant Skill

dominant\_skill mengambil skill pertama yang tersedia pada setiap lowongan sebagai representasi skill utama.

- Bersamaan dengan fungsi agregasi di atas, sistem mengambil nilai kompetensi pertama yang muncul pada baris lowongan melalui fungsi kuantifikasi ('skill\_name\_clean', 'first') untuk menghasilkan kolom baru bernama dominant\_skill. Variabel ini bertindak sebagai representasi keahlian utama dari sebuah posisi pekerjaan.

#### c. Skill text

- Langkah dan Alasan: Melalui fungsi ekspresi lambda values: ' '.join(sorted(set(values))), seluruh kumpulan kata kunci keahlian unik per baris pekerjaan digabungkan secara horizontal menjadi satu string teks panjang terurut dalam variabel baru bernama skill\_text.
- Evidence Data: Variabel ini sangat krusial bagi kebutuhan AI Engineer karena dapat digunakan langsung dalam proses pembentukan representasi matriks angka (vector embedding) untuk menghitung skor kemiripan semantik dengan CV pelamar. Nilai kosong pada kolom ini diisi dengan string kosong ""

skill\_text menggabungkan seluruh skill per lowongan menjadi string seperti python, sql, tableau, excel. Fitur ini sangat berguna untuk NLP, vectorization, dan matching CV.

```

if skill_ready_available:
 required_skill_columns = ('job_id', 'skill_name')
 missing_skill_columns = required_skill_columns - set(skill_ready.columns)

 if missing_skill_columns:
 print(f"Kolom skill tidak lengkap: {missing_skill_columns}. Fitur skill dilewati.")
 skill_features_available = False
 else:
 skill_ready['skill_name_clean'] = skill_ready['skill_name'].apply(clean_simple_text)
 skill_ready = skill_ready[skill_ready['skill_name_clean'] != ''].copy()

 skill_features = (
 skill_ready.groupby('job_id')
 .agg(
 total_skills=('skill_name_clean', 'nunique'),
 dominant_skill=('skill_name_clean', 'first'),
 skill_text=('skill_name_clean', lambda values: ' '.join(sorted(set(values))))
)
 .reset_index()
)

 jobs = jobs.merge(skill_features, on='job_id', how='left')
 jobs['total_skills'] = jobs['total_skills'].fillna(0).astype(int)
 jobs['dominant_skill'] = jobs['dominant_skill'].fillna('not_specified')
 jobs['skill_text'] = jobs['skill_text'].fillna('')
 skill_features_available = True

 print(f"Fitur skill berhasil dibuat untuk {skill_features.shape[0]} job_id unik.")
 display(skill_features.head())
else:
 skill_features_available = False
 print('skill_analysis_ready.csv tidak tersedia. Fitur total_skills, dominant_skill, dan skill_text dilewati.')

```

**\*\*Insight teknis:\*\*** Fitur skill adalah salah satu komponen terpenting untuk CareerPath AI. 'skill\_text' dapat langsung digunakan untuk perhitungan kemiripan teks antara CV dan lowongan, sedangkan 'total\_skills' membantu membaca kompleksitas kebutuhan kompetensi pada setiap job.

### 5.4 Feature Engineering Time

- Langkah dan Alasan: Data penanda waktu posting diekstrak secara aman menggunakan fungsi kustom parse\_datetime\_safe() berbasis library pd.to\_datetime(errors='coerce') pada kolom listed\_time atau original\_listed\_time. Dari objek waktu tersebut, saya memecah komponen tanggal menjadi tiga fitur waktu baru: posted\_year (tahun), posted\_month (bulan), dan posted\_dayofweek (hari dalam angka 0-6).
- Evidence Data: Variabel waktu ini secara akurat digunakan untuk menyajikan visualisasi tren dinamika waktu posting bulanan pada antarmuka aplikasi Streamlit.

Fitur waktu dibuat dari `listed_time` jika tersedia. Jika `listed_time` kosong, notebook mencoba menggunakan `original_listed_time`.

```
def parse_datetime_safe(series):
 """Mengonversi kolom tanggal menjadi datetime dengan aman."""
 return pd.to_datetime(series, errors='coerce')

if 'listed_time' in jobs.columns:
 posted_datetime = parse_datetime_safe(jobs['listed_time'])
elif 'original_listed_time' in jobs.columns:
 posted_datetime = parse_datetime_safe(jobs['original_listed_time'])
else:
 posted_datetime = pd.Series(pd.NaT, index=jobs.index)

if posted_datetime.isna().all() and 'original_listed_time' in jobs.columns:
 posted_datetime = parse_datetime_safe(jobs['original_listed_time'])

jobs['posted_year'] = posted_datetime.dt.year
jobs['posted_month'] = posted_datetime.dt.month
jobs['posted_dayofweek'] = posted_datetime.dt.dayofweek

jobs[['posted_year', 'posted_month', 'posted_dayofweek']].head()
```

### 5.5 Feature Engineering Location

- Langkah dan Alasan: Saya merancang fungsi pembersihan wilayah bernama `clean_city(location)`. Fungsi ini memotong string lokasi berbasis pembatas koma menggunakan logika `.split(',')[0]` untuk mengambil nama kota atau wilayah utama saja, membersihkan karakter non-alfanumerik, dan menyimpannya ke dalam fitur baru bernama `city_clean`. Selain itu, dibuat variabel boolean pendukung `is_unknown_location` bernilai 1 jika kota terdeteksi tidak jelas, dan bernilai 0 jika data kota tersedia valid.
- Evidence Data: Lokasi makro regional berhasil dipersempit areanya menjadi nama kota bersih, yang langsung bertindak sebagai filter dinamis di antarmuka platform.

## 7. Feature Engineering Location

Lokasi distandarisasi menjadi `city_clean`, dan dibuat indikator `is_unknown_location` untuk mendeteksi lokasi yang tidak diketahui.

```
def clean_city(location):
 """Mengambil kota utama dan menstandarisasi lokasi."""
 if pd.isna(location) or str(location).strip() == '':
 return 'unknown'
 city = str(location).split(',')[0].strip().lower()
 city = re.sub(r'^a-z0-9\s', '', city)
 city = re.sub(r'\s+', ' ', city).strip()
 return city if city else 'unknown'

jobs['city_clean'] = jobs['location'].apply(clean_city) if 'location' in jobs.columns else 'unknown'
jobs['is_unknown_location'] = np.where(jobs['city_clean'].isin(['unknown', 'not specified', 'not_specified']), 1, 0)

jobs[['location', 'city_clean', 'is_unknown_location']].head() if 'location' in jobs.columns else jobs[['city_clean', 'is_unknown_location']].head()
```

### 5.6 Final Dataset Check

- Langkah dan Alasan: Sebelum berkas diekspor, dilakukan validasi akhir (sanity check) menggunakan struktur data Pandas `pd.DataFrame()` untuk mengaudit ketersediaan seluruh variabel rekayasa fitur baru di dalam tabel utama.
- Evidence Data: Hasil kompilasi data membuktikan bahwa seluruh kolom baru (`title_clean`, `description_clean`, `desc_length`, `is_remote`, `experience_level_clean`, `posted_month`, `city_clean`, `total_skills`, `dominant_skill`, dan `skill_text`) telah berstatus Tersedia (True) dan melekat kokoh pada struktur data utama.

Tahap ini memastikan fitur utama telah terbentuk dan menampilkan preview dataset final.

```
expected_features = [
 'title_clean',
 'description_clean',
 'desc_length',
 'is_remote',
 'experience_level_clean',
 'posted_month',
 'city_clean'
]

if skill_features_available:
 expected_features.extend(['total_skills', 'dominant_skill', 'skill_text'])

feature_check = pd.DataFrame({
 'feature': expected_features,
 'tersedia': [feature in jobs.columns for feature in expected_features]
})

display(feature_check)
display(jobs.head())
```

## 5.7 Validasi Feature

- Langkah dan Alasan: Validasi akhir dijalankan dengan mengevaluasi ringkasan statistik deskriptif menggunakan perintah `.describe()` dan menghitung total persentase kekosongan data pasca-rekayasa fitur.
- Evidence Data: Hasil pemfilteran membuktikan tingkat missing value pada seluruh fitur baru adalah 0% (100% Terisi Sempurna). Statistik deskriptif pada fitur `experience_level_clean` membuktikan data berhasil terisi penuh tanpa ada baris penting yang terbuang, dengan dominasi level Mid. Metrik `desc_length` dan `total_skills` menunjukkan distribusi data yang variatif dan sehat.

Validasi fitur dilakukan pada missing values fitur baru, distribusi `experience_level_clean`, statistik `desc_length`, dan statistik `total_skills` jika tersedia.

```
new_features = [
 'title_clean',
 'description_clean',
 'desc_length',
 'is_remote',
 'experience_level_clean',
 'posted_year',
 'posted_month',
 'posted_dayofweek',
 'city_clean',
 'is_unknown_location'
]

if skill_features_available:
 new_features.extend(['total_skills', 'dominant_skill', 'skill_text'])

feature_missing = pd.DataFrame({
 'feature': new_features,
 'missing_count': jobs[new_features].isna().sum().values,
 'missing_percent': (jobs[new_features].isna().mean() * 100).round(2).values,
 'type_data': jobs[new_features].dtypes.astype(str).values
})

display(feature_missing)

print("Distribusi experience_level_clean:")
display(jobs['experience_level_clean'].value_counts().reset_index().rename(columns={'index': 'experience_level_clean', 'experience_level_clean': 'jumlah'}))

print("Statistik desc_length:")
display(jobs['desc_length'].describe())

if skill_features_available:
 print("Statistik total_skills:")
 display(jobs['total_skills'].describe())
```

## 5.8 Simpan output final

- Langkah dan Alasan: Tahap akhir dari pipa rekayasa fitur ini adalah menyimpan seluruh objek data yang telah kaya fitur ke dalam penyimpanan lokal menggunakan fungsi bawaan Pandas `.to_csv()` tanpa menyertakan indeks.



- Evidence Data: Dataset final berhasil diekspor secara sempurna dengan nama berkas resmi job\_featured.csv. Dokumen ini menjadi Gold Dataset steril yang siap dikonsumsi langsung oleh sistem rekomendasi kecerdasan buatan

```
Dataset final disimpan sebagai job_featured.csv.

OUTPUT_PATH = 'job_featured.csv'
jobs.to_csv(OUTPUT_PATH, index=False)

print(f'Dataset feature engineering berhasil disimpan ke: {OUTPUT_PATH}')
print(f'Shape final: {jobs.shape[0]:,} baris x {jobs.shape[1]:,} kolom')
```

## 5.9 Dokumentasi Feature Dictionary

Berikut adalah tabel dokumentasi kamus fitur (feature dictionary) berdasarkan struktur data akhir pada variable

| Feature                | Tipe Data         | Deskripsi Fitur                                                                |
|------------------------|-------------------|--------------------------------------------------------------------------------|
| Title_clean            | Object (string)   | Title pekerjaan yang telah dibersihkan untuk kebutuhan NLP dan matching.       |
| Description_clean      | Object (string)   | Deskripsi pekerjaan yang telah dirapikan untuk NLP dan semantic matching       |
| Desc_length            | Int64 (numerik)   | Jumlah kata pada description_clean; indikator detail dan kompleksitas lowongan |
| Is_remote              | Int64 (biner)     | Indikator pekerjaan remote; 1 remote, 0 non-remote atau unknown.               |
| Experience_level_clean | Object (string)   | Level pengalaman standar hasil normalisasi dari experience level dan title.    |
| Total_skills           | Int32 (numerik)   | Jumlah skill unik yang terkait dengan job_id.                                  |
| Dominant_skill         | Object (string)   | Skill utama atau skill pertama yang terkait dengan job_id                      |
| Skill_text             | Object (string)   | Gabungan seluruh skill per job dalam format teks untuk NLP dan CV matching     |
| Posted_year            | Float64 (numerik) | Tahun posting lowongan pekerjaan.                                              |
| Posted_month           | Float64 (numerik) | Bulan posting lowongan pekerjaan.                                              |
| Posted_dayofweek       | Float64 (numerik) | Hari posting dalam angka; 0 Senin sampai 6 Minggu                              |
| City_clean             | Object (string)   | Nama kota/lokasi utama yang telah distandarisasi.                              |
| Is_unknown_location    | Int64 (biner)     | Indikator lokasi unknown; 1 unknown, 0 tersedia.                               |

#### 5.10 Kesimpulan Akhir Feature Engineering

Rangkaian eksperimen rekayasa fitur pada berkas `feature_engineering_final.ipynb` telah berhasil mentransformasikan data mentah menjadi bentuk biner, numerik, dan teks bersih terstandarisasi secara utuh. Variabel kontekstual seperti `title_clean`, `description_clean`, dan `skill_text` bertindak sebagai asupan data utama bagi tim AI Engineer untuk melakukan tokenisasi dan pembentukan representasi matriks angka (vector embedding) menggunakan arsitektur IndoBERT. Metrik pelengkap seperti `total_skills`, `desc_length`, `is_remote`, dan `experience_level_clean` memberikan fleksibilitas penuh untuk menyusun operasi logika matematika filtering dan menyajikan visualisasi data yang interaktif pada antarmuka aplikasi Streamlit di dunia nyata.

## **BAB VI**

### **REKOMENDASI**

#### **6.1 Rekomendasi action item**

Berdasarkan hasil temuan data di atas, berikut Adalah rekomendasi pengembangan untuk platform CareerPathAI

1. Action Item 1: Rekayasa Ekstraksi Teks (Solusi Masalah Data Kosong 72%)  
Rekomendasi: Tim AI Engineer harus membangun fungsi Heuristic Labeling berbasis Keyword Matching pada teks deskripsi pekerjaan. Jika sistem mendeteksi kata kunci seperti "junior", "intern", atau "senior" pada teks deskripsi, sistem harus otomatis mengimputasi level pengalaman tersebut ke kolom baru (`experience_level_clean`) untuk mengisi kekosongan data bawaan platform yang sebesar 72% tadi.
2. Action Item 2: Penggabungan Variabel Fitur Tunggal (Feature Combination)  
Rekomendasi: Lakukan penggabungan string antara judul bersih dan deskripsi bersih menjadi satu fitur tunggal (`df['cleaned_features'] = df['title_clean'] + ' ' + df['description_clean']`) sebelum teks diubah menjadi vektor oleh IndoBERT. Teknik rekayasa fitur ini terbukti secara ilmiah mampu mencegah Data Leakage (kebocoran data) dan melipatgandakan akurasi sistem rekomendasi.
3. Action Item 3: Pengembangan Fitur Interaktif pada Dashboard Streamlit  
Rekomendasi: Dashboard Streamlit wajib dilengkapi dengan filter dinamis berbasis 15 besar rumpun keahlian industri (Top 15 Skills) serta indikator fleksibilitas kerja (`is_remote`). Fitur ini akan sangat membantu pengguna untuk memfilter lowongan secara personal sekaligus melihat letak kesenjangan skill mereka secara langsung melalui dashboard.